T-Press Documentation

A hands-free VR interaction with the tongue

Updated 22/05/2026

1. Introduction

What is T-Press?

T-Press is a hands-free VR interaction system that uses the OpenBCI Cyton Board and EMG sensors to detect tongue gestures. The demo is set in a smart room environment where the user can interact with different objects in VR using tongue movements. These interactions include opening and closing drawers, opening doors, turning on the TV, and controlling the lights.

What can it be used for?

It can be used to perform hands-free gestures in VR with the tongue while the user is seated.

How does it work?

The prediction of gestures is based on a model trained individually for each user, detecting left, front or right, as well as a gradient of pressure in each of the sides from 0 to 100. The system uses three main devices: a Meta Quest 2 or Meta Quest 3 headset, the OpenBCI sensors, and a computer. The computer acts as an intermediary device, handling communication between the VR headset and the sensors.

What are the prerequisites to work with this demo?

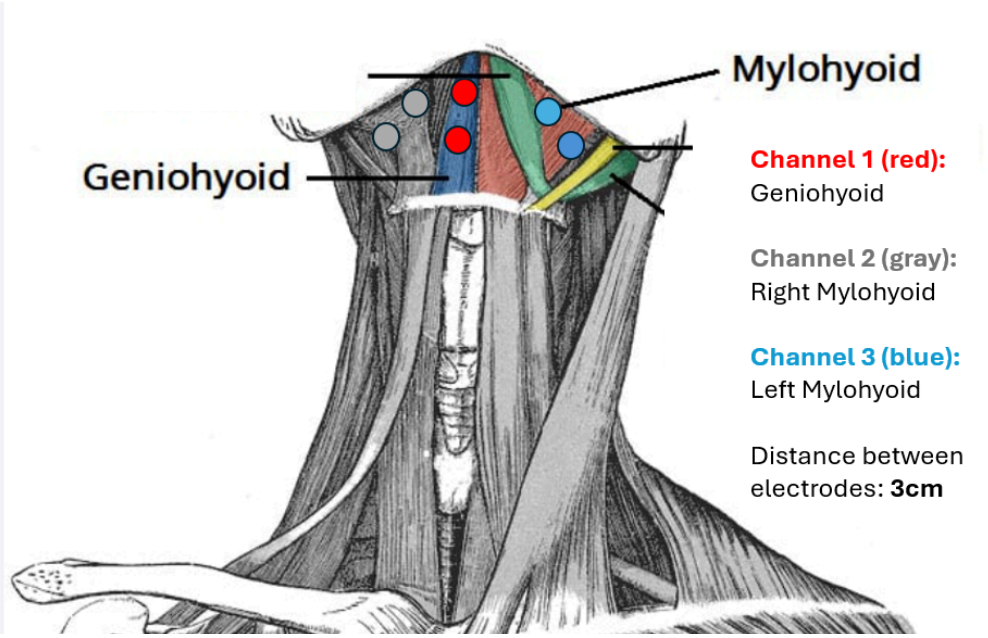
Having a MetaQuest 2 or 3, the OpenBCI Cyton board and a computer with a Python environment. Ideally, to extend the demo it is necessary to have basic knowledge about Unity, machine learning and Python.

2. OpenBCI board and GUI

What is OpenBCI?

T-Press uses the OpenBCI Cyton Board to detect voltage changes in the tongue muscles through EMG signals. Each sensor is connected to two bipolar electrodes, which are typically purchased separately. The Cyton Board is an open-source biosensing board commonly used for EMG, EEG, and other bioelectric signal acquisition. The board can be configured using the OpenBCI GUI.

In total, one reference sensor and three sensors are placed on the submental triangle. Each sensor uses two bipolar electrodes. The image below shows the electrode placement positions.

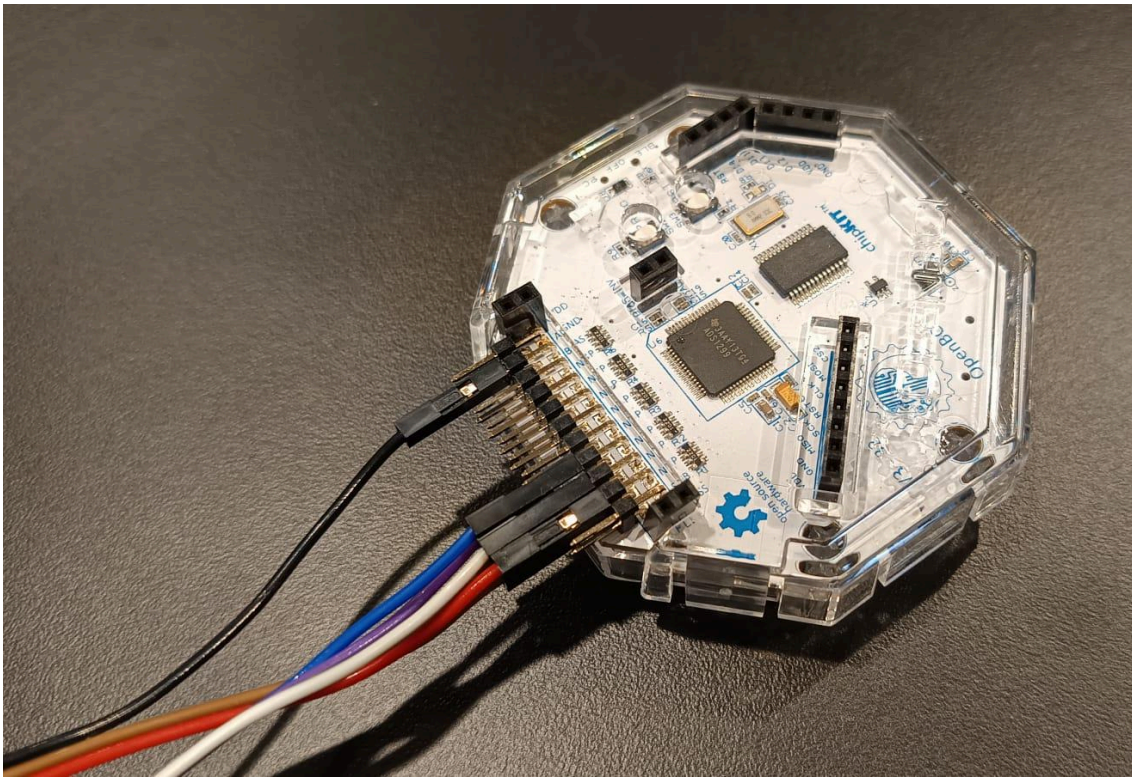


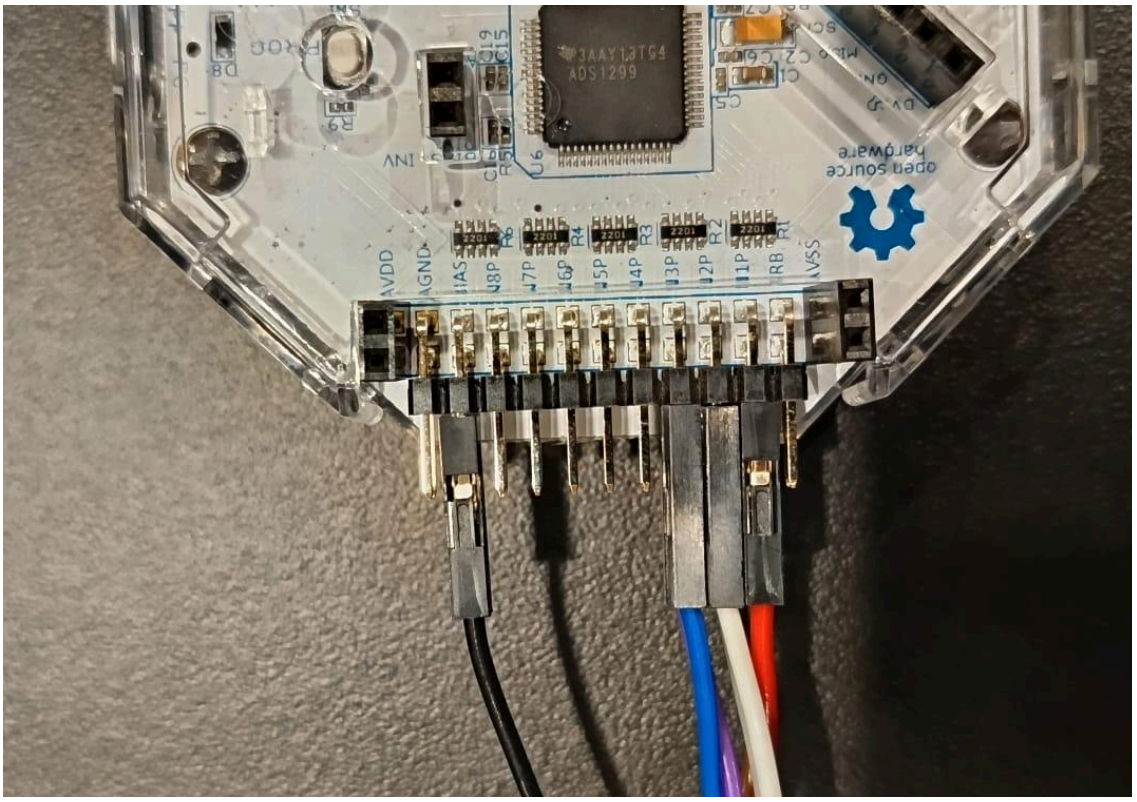
How to connect the sensors

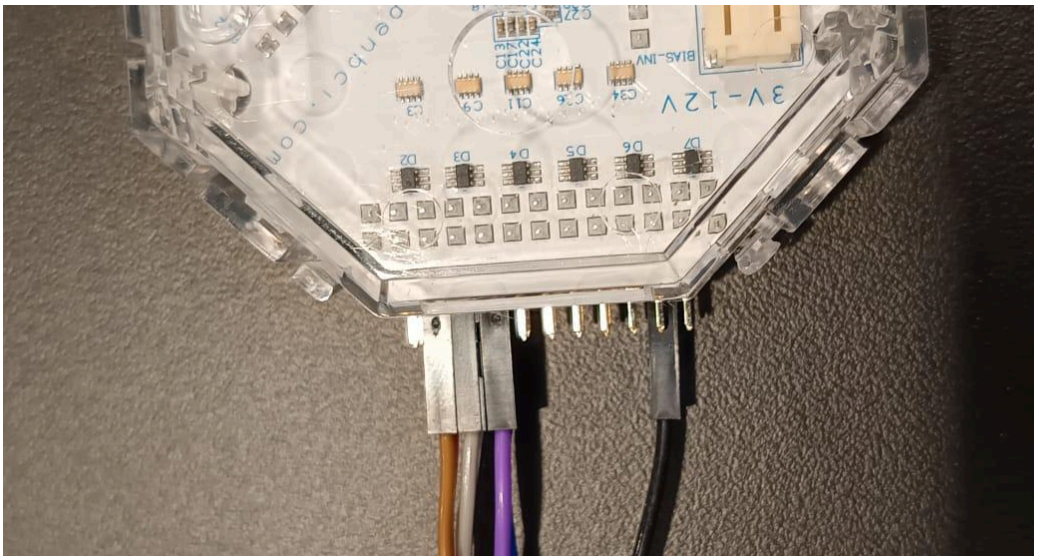
The first step in working with the T-Press demo is connecting the sensors and familiarising with the OpenBCI GUI.

1. If it is the first time running the demo, if the OpenBCI GUI is not installed in your computer, or if you just want to check the board for the first time, complete the official Getting Started guide from OpenBCI.
2. After the guide is completed, connect the sensors to the board like this:
    a. **Black** wire to **BIAS**

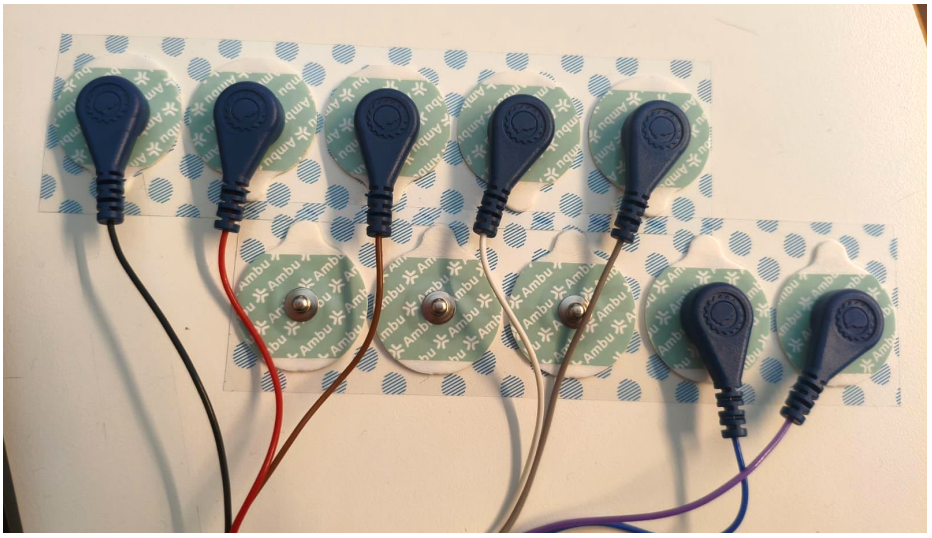
b. **Red and Brown** wires to both `N1P.` This will be Channel 1.
c. **White and Gray** wires to both `N2P.` This will be Channel 2.
d. **Blue and Purple** wires to both `N3P.` This will be Channel 3.





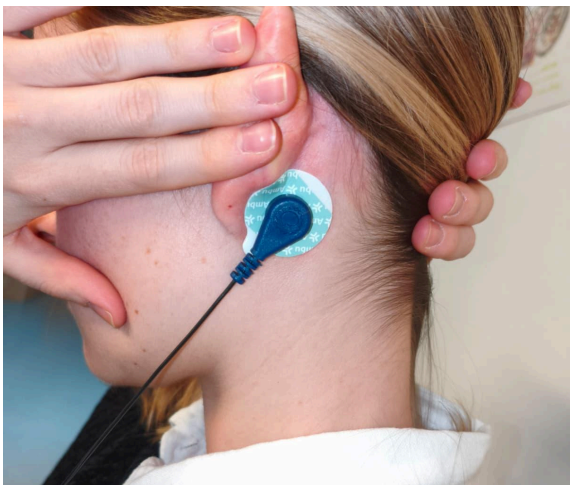


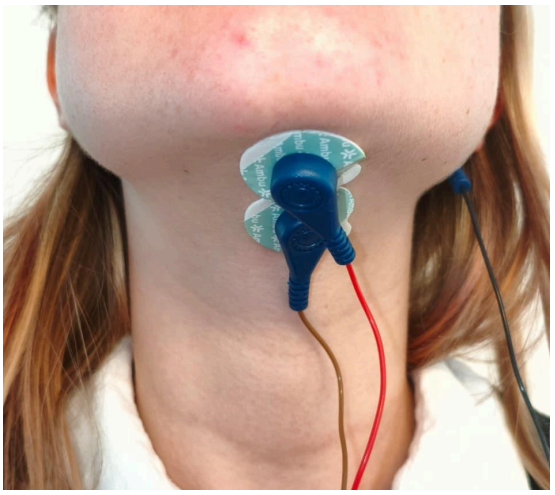
3. To each sensor, attach an electrode. The recommended electrodes used for the demo are **Ambu WhiteSensor WSP30**.

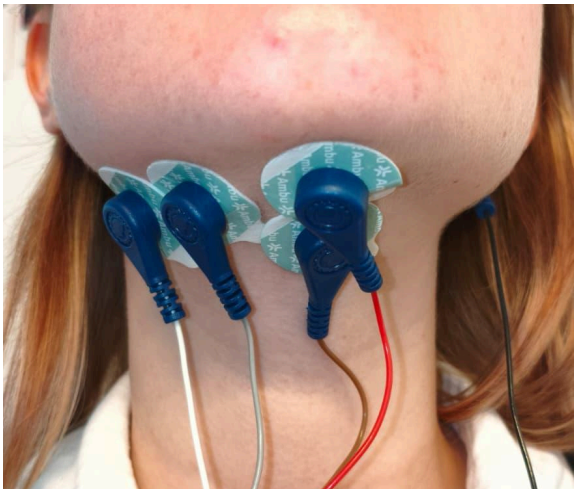


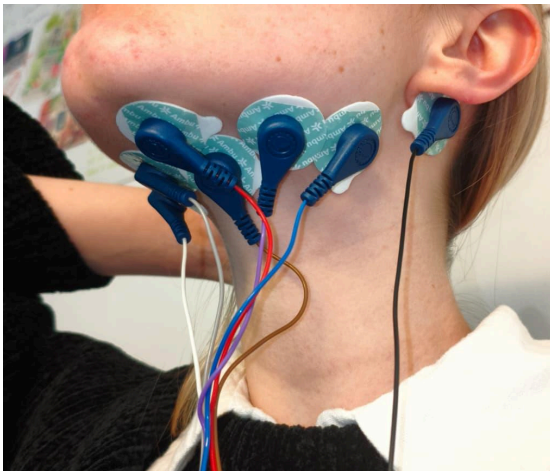
4. Clean the submental triangle and the area behind your left ear with an alcohol-based wipe. **Important**: the sensors do not work if there is facial hair.
5. Now place the electrodes in your skin. See the photos in the next page for reference. **Suggestion**: if you're placing the electrodes yourself, a mirror or the phone's front camera can be helpful.
    a. Place 1 bias electrode on your left mastoid as a reference. The wire should be **Black**.
    b. Place 2 electrodes for N1P (**Red and Brown** in the pictures, Channel 1) in the **center** of the submental triangle.
    c. Place 2 electrodes for N2P (**White and Gray** in the picture, Channel 2) on the **right** side of the submental triangle.
    d. Place 2 electrodes for N3P (**Blue and Purple** in the picture, Channel 3) on the **left** side of the submental triangle.

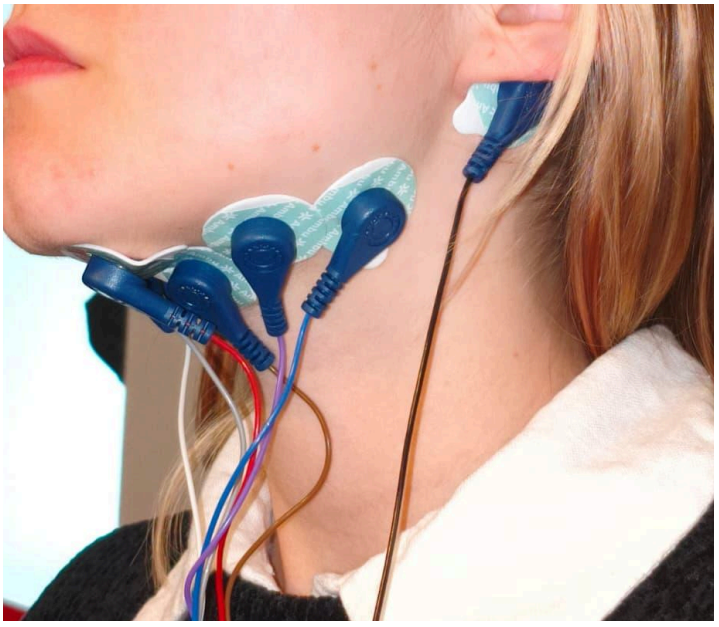
6. Now, continue to the following section to configure the board and start using the demo.











3. Starting the T-Press demo

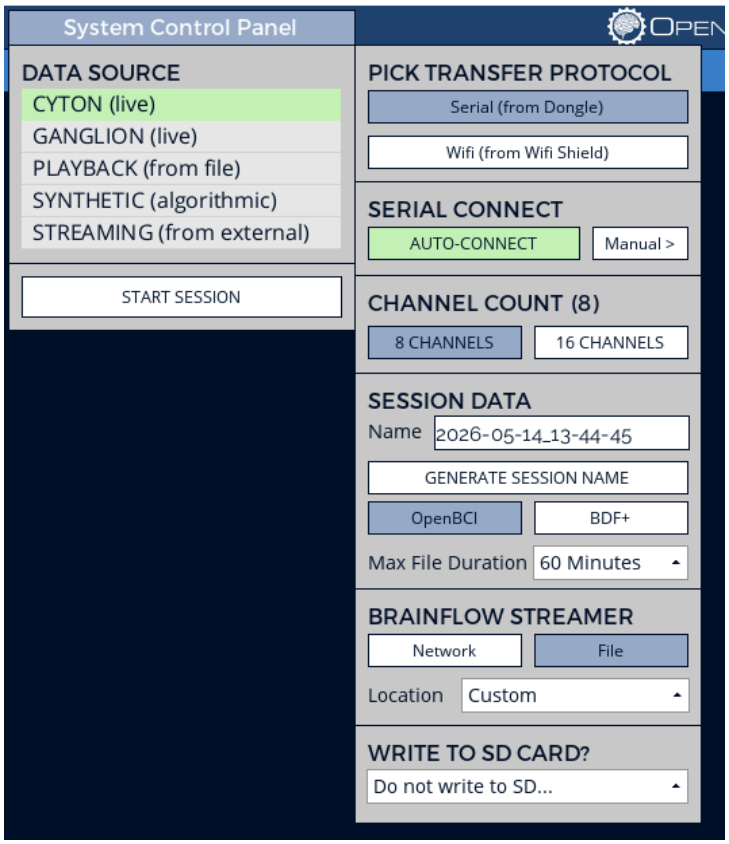
Step 1: First time running the demo

This is your first time running the demo, so the Unity project needs to be installed in the VR headset and the python environment prepared to run the classification pipeline. This step only needs to be done once.

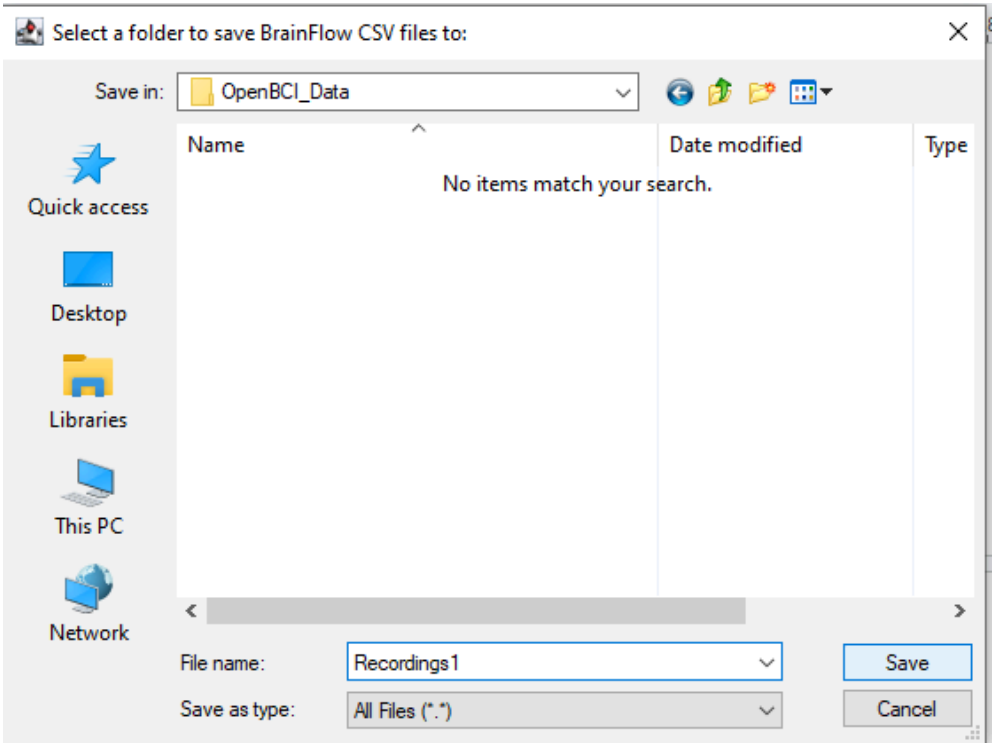
1. Prepare your preferred environment for running Python code locally on your computer. The author used Anaconda on Windows 10, but any Python environment can be used.
2. Install the python dependencies in `SmartRoom / Pressure_Classifier / requirements.txt`.
3. Install this Unity version: **6000.2.14f1**.
4. Download the Unity project from GitHub HyperDecahedron/SmartRoomVR (main branch).
5. Deploy the Unity project into the Meta Quest 2 or 3.

Step 2: Configure OpenBCI GUI environment

1. Turn on the board (“PC”, not “BLE”), connect the dongle to a USB port in your computer and open OpenBCI GUI.

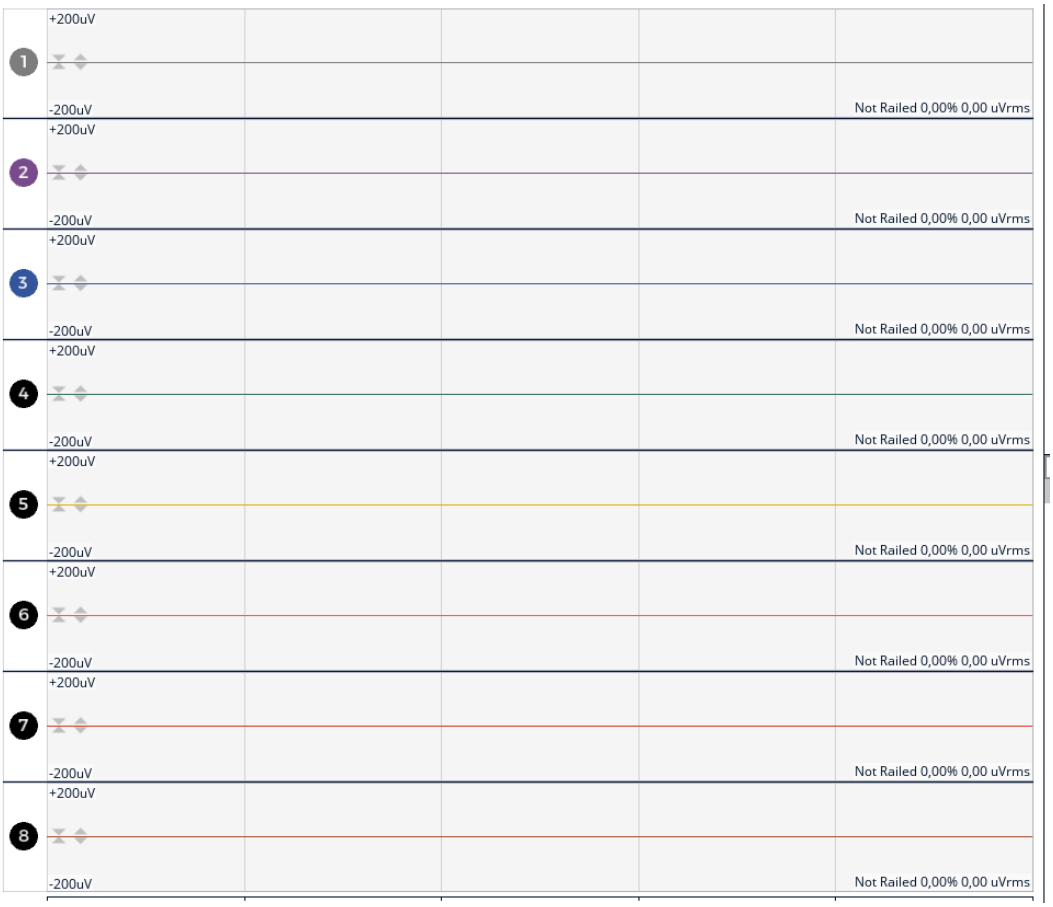


2. Configure the session with the settings as in the picture above. Click **Location - Custom** to select a custom path to save the recordings. In the explorer window that will pop-up, navigate to `SmartRoom / Pressure_Classifier / OpenBCI_Data.` **Important**: Once inside `OpenBCI_Data,` set the **file name** to `Recordings{user}.` Don't add the brackets (for example, Recordings1). Don't add a new folder, just change the file name as in the picture below.



`user` is a number that will represent the user in this session. You can choose the number that you want. Remember this number, since it will be relevant in next steps. For example, if `user` is '1', the custom path should be `SmartRoom / Pressure_Classifier / OpenBCI_Data / Recordings1.` Click **Save**.

3. Click **AUTO-CONNECT**.
4. Prepare the board and sensors as described in **Section 2 - OpenBCI board and GUI**.
5. Set off all channels except for channels 1, 2 and 3. To do this, click on the icon of each number 4, 5, 6, 7, 8 so they look like in the picture in the next page.
6. Open Hardware Settings and set **SRB2 Off** for channels 1, 2 and 3. Click '**Send**' to send the settings to the board.
7. Click on **TimeSeries** to go back to the visualisation tab.
8. Click on **Start Data Stream**.



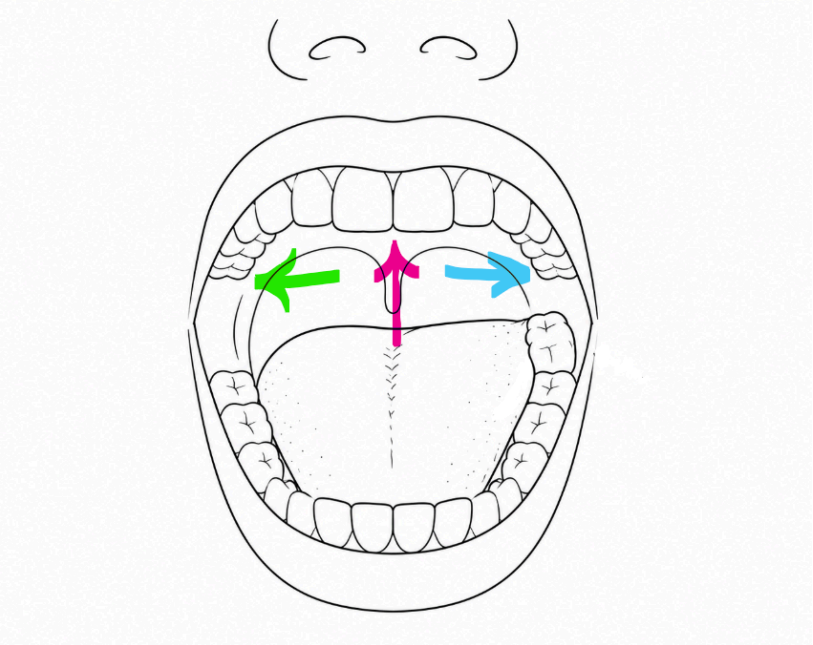
Step 3: Train the model

1. Make sure both the Meta Quest and the laptop are connected to the same network, preferably a local network or a mobile hotspot. No internet connection is required.
2. Now, you will start the automated classification pipeline to train the model to your individual movements and perform the classification in real time. During this process, you will be prompted to execute certain movements with your tongue. You will train four different tongue positions: `Rest`, `Left`, `Front`, and `Right`. To record a label, move your tongue to the specified position and maintain steady pressure until the UI indicates that the recording is complete. Each position takes approximately 15 seconds to record. Your mouth should remain closed during the entire process. To record each position:
    a. **Rest:** Place your tongue in a neutral position and avoid moving it or swallowing.
    b. **Left:** Move the tip of your tongue to the last molar on the upper-left side and press from behind.
    c. **Front:** Press your tongue firmly upward against the palate.

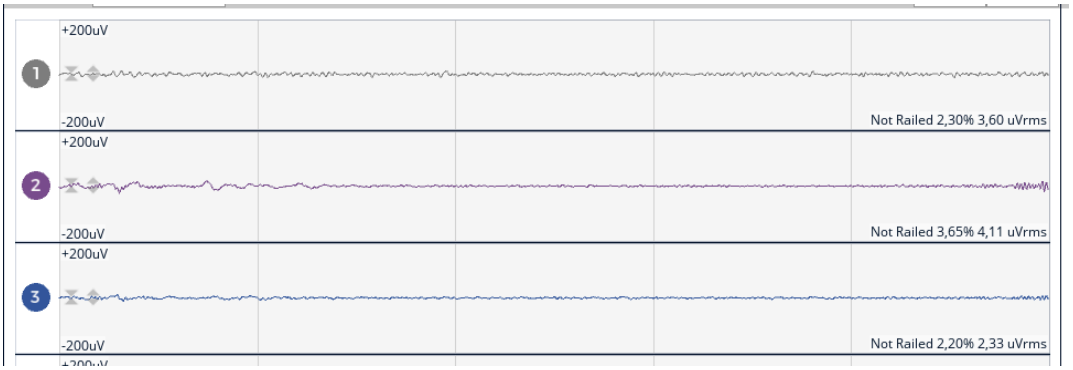
d. **Right:** Move the tip of your tongue to the last molar on the upper-right side and press from behind.

Take some time to practice each position.

Additionally, the tongue pressure on each of the sides (Left, Front, Right) will also be considered. You will train two different pressures `50%, 100%`. The pressure `50%` represents approximately half of your maximum pressure on the corresponding side. The pressure `100%` represents your maximum pressure. This way, you will train sides and pressures in combination (`Left 50%, Left 100%, Front 50%...`).



3. **Important:** Before starting the automatic classification pipeline, you should check the noise levels of the tongue pressure signal. Open the OpenBCI GUI and go to your rest position. The signal should look similar to this:

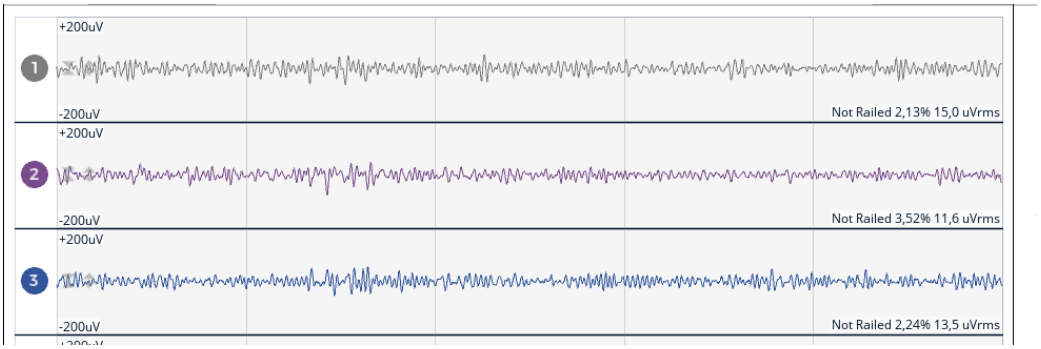


The signal will vary a lot from user to user. A user may have a noisy resting position, and that is alright if the other positions also have higher amplitudes. The important aspect is being able to tell the difference between the resting position and the other positions. Additionally, the less noise, the better. If the signal is noisy, check **Section 9 Troubleshooting - Noisy Signal in OpenBCI**.

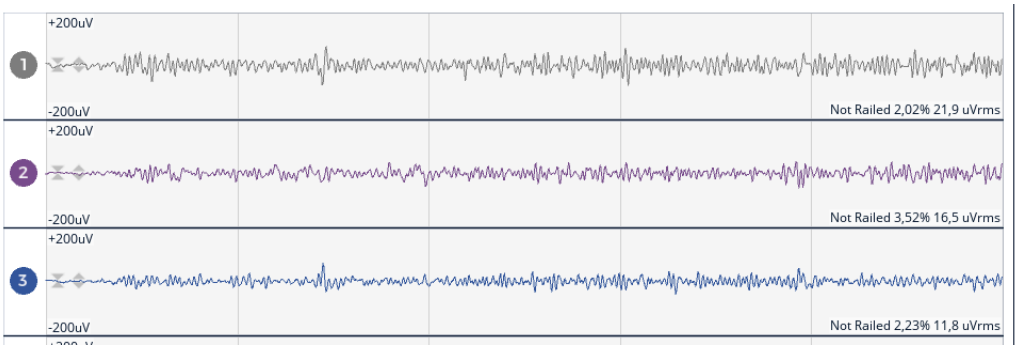
It is very important to avoid moving your body during the training process, since those movements may be detected by the model. Try not to swallow during the resting position training.

These are examples of how the signal should look for different tongue positions for one user (it may look different for you).

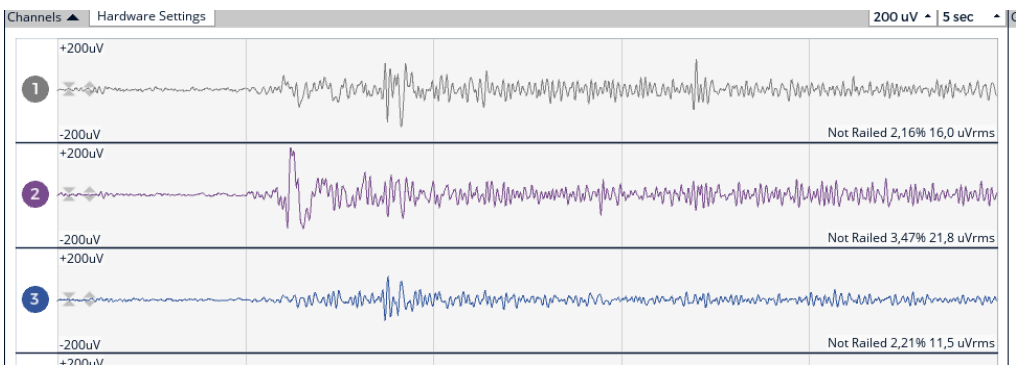
LEFT 100%



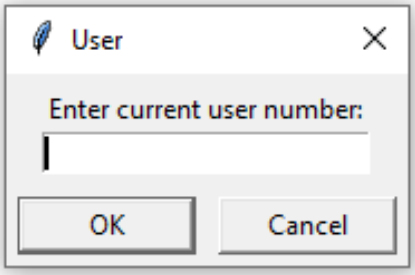
FRONT 100%

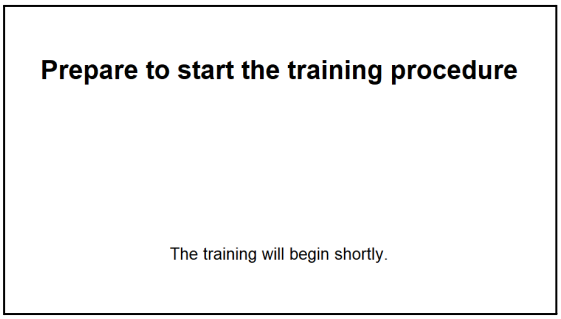


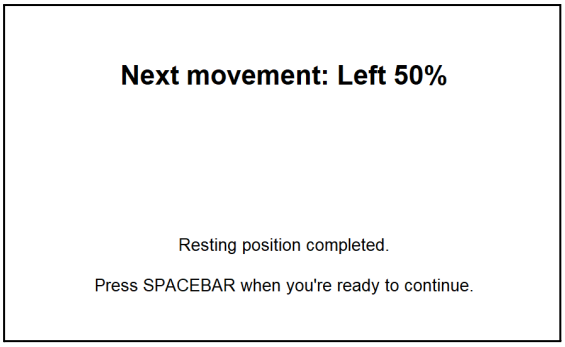
RIGHT 100%

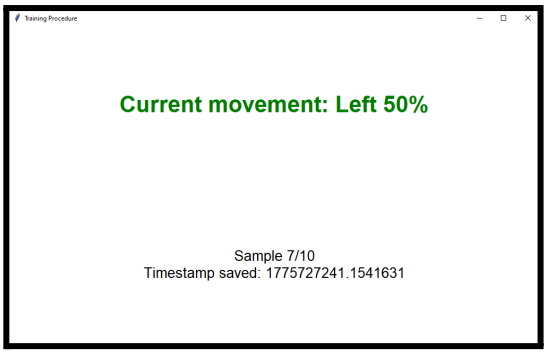


4. Launch the script `CLASSIFICATION_PIPELINE.py` in `SmartRoom / Pressure_Classifier.` **Continue to the next step before doing anything in the UI that will appear**.
5. After launching the script, a pop-up window will appear. Write the same number for the user that you used in **Step 2.2**. For example, '1'. **Important**: be ready, when you enter the number, the training pipeline will start. Follow the instructions on the screen to record samples of each class. When the text becomes green, it means the tongue pressure is being recorded. See examples of how the UI will look like below.

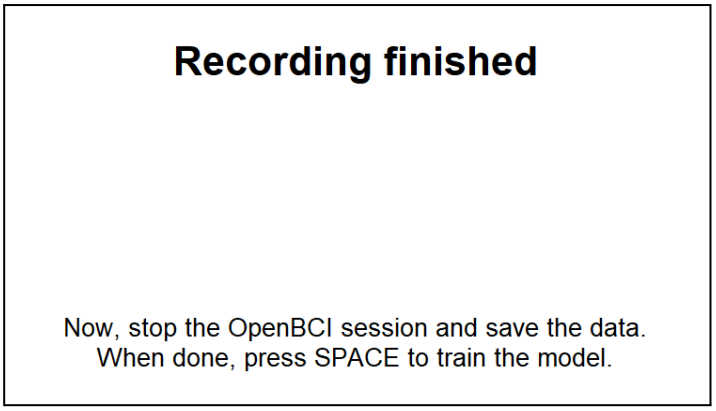




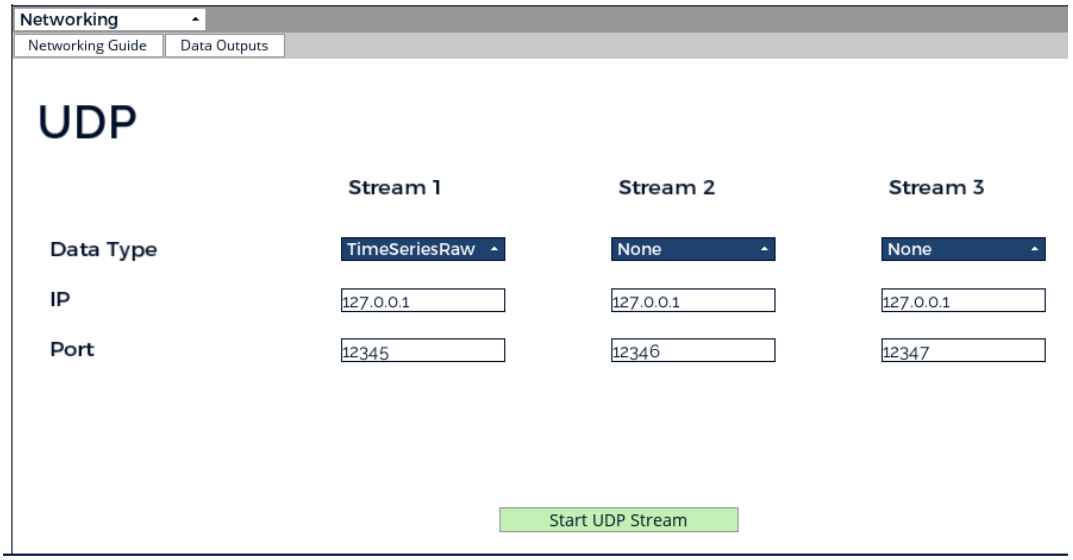




6. When the text says "**Recording finished**", don't close the window. Go to OpenBCI GUI and click **Stop Data Stream**, then click **System Control Panel - Stop Session.** This will stop the OpenBCI session and save the data automatically.



7. In the training procedure window, press SPACE to train the model and wait until it finishes. It will take approximately 1-2 minutes.
8. When the model finishes training, the text will say "**Now, start the OpenBCI session with UDP**". To do so, go to OpenBCI GUI, create a new session and configure the environment as in **Step 2**. The name of the file is not relevant this time, you can select the path and name that you prefer.
9. Make sure the Meta Quest and the computer are in the same network, preferably a local network or a mobile hotspot. No internet connection required.
10. In the right side of OpenBCI GUI, change one of the two tabs to **Networking** and select **UDP** and **Stream 1 - TimeSeriesRaw**. Click on **Start UDP Stream**.



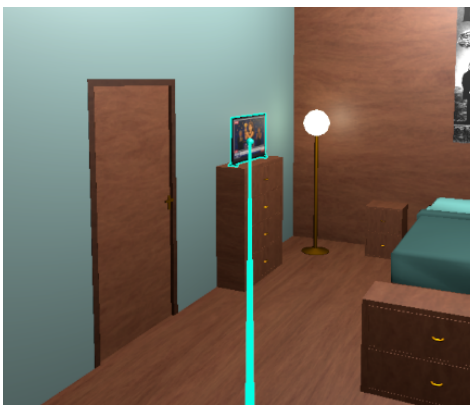
11. Go back to the training procedure, and press SPACE to continue.
12. A pop-up window will appear to write the **Quest IP**. This is the IP address of the Meta Quest with the deployed Unity project. Write the IP and click OK.

13. The **online classification will start**. This means that the model running in the computer will start processing tongue data and sending the predicted values to the Meta Quest. Continue to the next step to start using the demo in the Meta Quest.

Step 4: Execute the VR demo

As a result of the previous step, the gesture detection pipeline in the computer will be running and detecting your gestures.

1. Put on the MetaQuest and open **SmartRoomVR**.
2. You can now start using your tongue to interact with the VR environment. Press firmly on the side that feels most comfortable to trigger a “click.” Move your head to highlight objects, which will turn blue when they are ready to be interacted with. Try the following interactions:
    a. Open/Close the door.
    b. Turn on/off the TV.
    c. Open/Close some drawers.
    d. Turn on/off the lights.
3. There is a cooldown of 3 seconds for each click with the tongue.
4. If you experience difficulties pressing with the tongue, test with different sides and pressures.
5. The tongue clicks are detected when there is a raise in the tongue pressure. Thus, it is necessary to go to your resting position in between clicks.

4. Training and gestures predictor

The model is trained using EMG signals collected from tongue muscle activity through the OpenBCI Cyton Board. This data is automatically saved when the session is stopped in the OpenBCI GUI. You can learn more about the data format at the end of this website: [The OpenBCI GUI Documentation].

The data used in this demo consists of EXG Channel 0, EXG Channel 1, EXG Channel 2, and the Timestamp. The timestamp indicates the exact time at which each sample was recorded. Each cell represents the voltage detected during the session.

However, to train the model, it is not possible to use the entire continuous signal directly. Instead, specific windows of data must be extracted that correspond to the moments when the user was performing the resting, left, front, or right tongue positions. To achieve this, the automatic classification pipeline also saves a timestamp file containing the exact recording time for each sample.

The tongue pressure data is ideally stored in:

`SmartRoom/Pressure_Classifier/OpenBCI_Data/Recordings{user}`

The timestamp data is stored in:

`SmartRoom/Pressure_Classifier/timestamps`

Once these two files are prepared and saved in the correct locations, the script `CLASSIFICATION_PIPELINE.py` calls another script named `Model_trainer.py`. This script performs the following steps:

- Splits the tongue pressure data into windows of 1.25 seconds, obtaining 10 windows per class (resting, left 50%, left 100%, etc.).
- For each window, additional overlapping windows are extracted every 0.2 seconds within the original window. This results in 6 additional windows for each base window.
- In total, the dataset contains 60 samples per class.

Each window is filtered to reduce noise using the following filters:

- Bandpass filter: 5–120 Hz
- Notch filter: 50 Hz
- Hilbert transform

The next step is feature extraction. The following features are computed from each window:

- RMS
- RMS signed difference
- Zero crossings
- Waveform length
- Maximum absolute value

- `np.std`
- `np.var`
- IAV
- Mean frequency

The data is then split into training and testing groups.

The model is trained using feature scaling and a parameter grid search. The final model is a `MultiOutputRegressor` based on a Random Forest regressor. The model predicts pressure values independently for the left, front, and right tongue positions, using the following format:

```
[left pressure [%], front pressure [%], right pressure [%]]
```

For example:

```
[25, 0, 0]
```

represents a pressure value of 25% on the left side.

5. Tongue interface in Unity

In the T-Press demo, Unity is completely unaware of the gesture prediction process, since it only listens to the gestures published by the computer. This allows the VR headset to focus only on rendering the VR scene and receiving the detected gestures through a UDP port.

To detect tongue gestures and convert them into actions inside the smart room, three main scripts are used. Note that the Unity scene is located under `Scenes/Smart_Room`.

TongueInterface.cs

This script handles communication between the computer and Unity using UDP networking. It starts a UDP listener on port `5052` and continuously receives data from the external tongue detection model running in Python on the computer. Both the VR headset running the demo and the laptop must be connected to the same Wi-Fi network.

The data is received in the following format:

```
(jitter, left pressure [%], front pressure [%], right pressure [%])
```

Data	**Type**
jitter	float (0 to 1)
left pressure	int (0 to 100)
front pressure	int (0 to 100)
right pressure	int (0 to 100)

The `jitter` is not used in the T-Press demo for simplicity purposes. To detect a click on the tongue, the script looks at the highest pressure in any of the left, front or right sides.

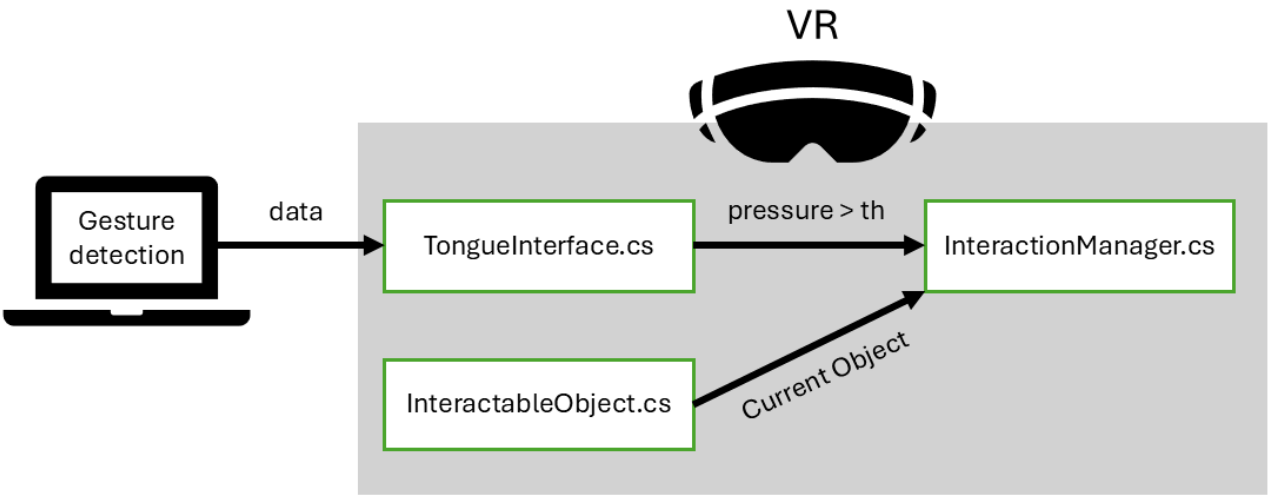
Each frame, the script checks the latest tongue pressure values and determines whether any pressure exceeds a predefined threshold. If the threshold is reached, the system interprets this as a tongue click and triggers an interaction through the `InteractionManager`. A cooldown period is applied before another tongue gesture can trigger a new interaction.

InteractionManager.cs

This script contains the functions used to interact with the smart room objects, including drawers, lights, the TV, and the door. It contains a public property called `current_object`. This property is updated whenever the user highlights a new interactable object. Calling `SelectObject()` from `TongueInterface.cs` automatically interacts with the currently selected object.

InteractableObject.cs

This script is attached to every interactable object in the scene. It detects when the head raycast collides with the object. When this happens, the object updates the `InteractionManager` to indicate that it is the currently highlighted object.



6. How to use the keyboard

The keyboard can be used to interact with the T-Press demo for debugging purposes or as an additional fallback input method. To use the keyboard, follow these steps:

1. Open the script `Keyboard_UDP_sender.py` in `SmartRoom \ Pressure_Classifier.`
2. Change the variable called `UNITY_IP` to the IP of your Meta Quest. If you are just debugging in Unity with a computer, you can also set the IP to `127.0.0.1` to send the information to the project in the same computer.
3. Use the keypad in your keyboard to send information. See the reference below.

Keypad		
7 Left pressure of 90/100 Jitter: 0.9 Data sent: (0.9, 90, 0, 0)	**8** Front pressure of 90/100 Jitter: 0.9 Data sent: (0.9, 0, 90, 0)	**9** Right pressure of 90/100 Jitter: 0.9 Data sent: (0.9, 0, 0, 90)
4 Left pressure of 60/100 Jitter: 0.6 Data sent: (0.6, 60, 0, 0)	**5** Front pressure of 60/100 Jitter: 0.6 Data sent: (0.6, 0, 60, 0)	**6** Right pressure of 60/100 Jitter: 0.6 Data sent: (0.6, 0, 0, 60)
1 Left pressure of 8/100 Jitter: 0.25 Data sent: (0.25, 8, 0, 0)	**2** Front pressure of 8/100 Jitter: 0.25 Data sent: (0.25, 0, 8, 0)	**3** Right pressure of 8/100 Jitter: 0.25 Data sent: (0.25, 0, 0, 8)

7. How to extend the code

How to include the Jitter

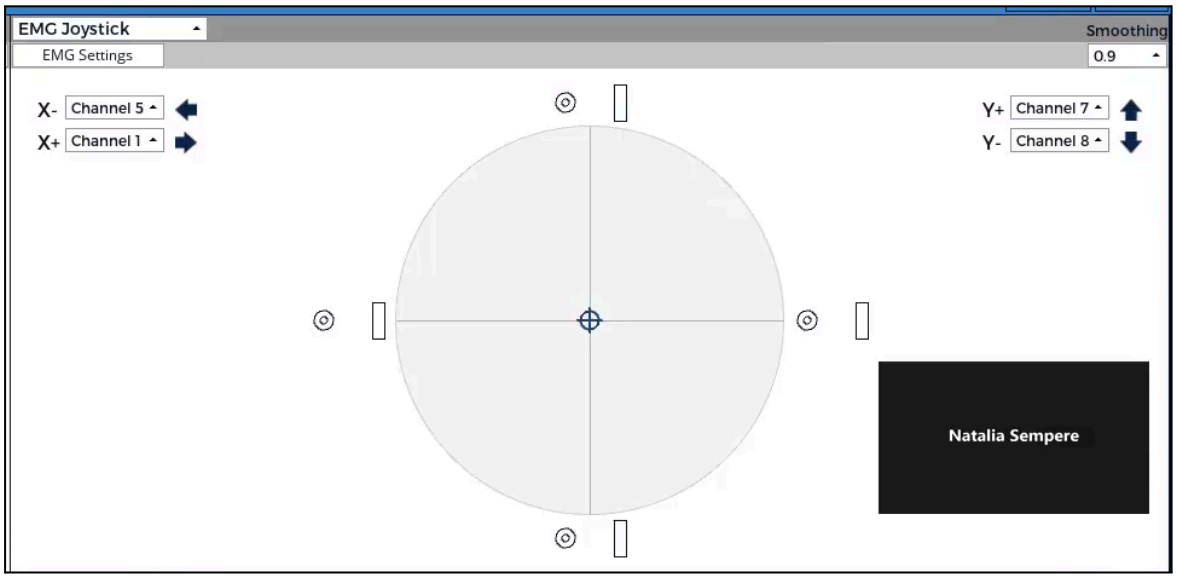
The jitter is an additional measure of the tongue interaction that indicates the raw effort of the tongue as measured by the sensors without further filtering. This measure can be useful for reflecting the raw tongue effort in objects within the virtual world.

For example, in a game about catching wasps, the jitter could be used to modify the behaviour of the wasps, making them more “angry” as the jitter increases. Another example is a car driving game, where the jitter could modify the vibration of the car, making it shake more as the tongue effort becomes more intense. This allows the user to see their effort reflected in the virtual environment.

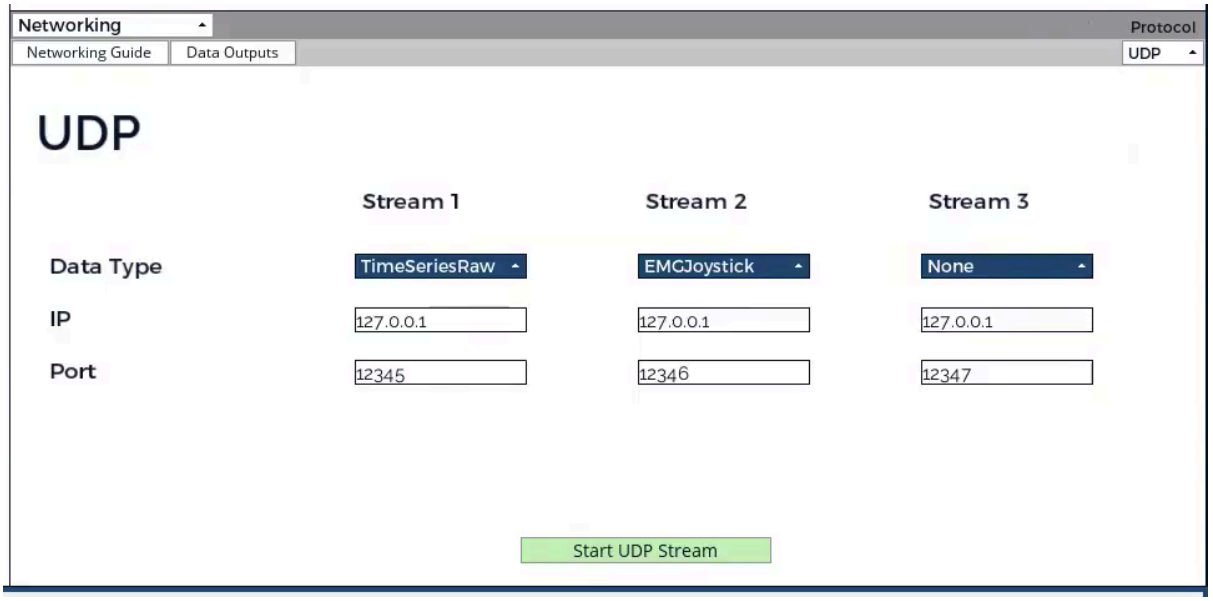
The jitter is already sent to Unity by default. In the `TongueInterface.cs` script, it is stored together with the pressure values from the left, front, and right tongue sensors. However, for the jitter to be detected from the main online classification pipeline, it must first be enabled in the OpenBCI GUI. Otherwise, Unity will always receive a jitter value of `0.0`.

To activate the jitter in the OpenBCI GUI, follow these steps:

1. Open the script `Online_classification.py` in `SmartRoom / Pressure_classifier.`
2. Uncomment the line `#jitter = get_data2()` and comment the following line `jitter = 0.0.`
3. During the usual training procedure (Section 3), you will need to add a second UDP steam. To do so, follow the training procedure until step 3.3.9. Then, open the **EMG Joystick tab in** the OpenBCI GUI.



4. Set `X+` to the channel you want to monitor. For example, if you want to receive the jitter from Channel 1, configure the channels as follows:
    - `X-`: Channel 5
    - `X+`: Channel 1
    - `Y-`: Channel 6
    - `Y+`: Channel 7
5. Open the **Networking** tab.
6. Configure **Stream 1** with the following settings:
    - **Data Type:** `TimeSeriesRaw`
    - **IP:** `127.0.0.1`
    - **Port:** `12345`
7. Configure **Stream 2** with the following settings:
    - **Data Type:** `EMGJoystick`
    - **IP:** `127.0.0.1`
    - **Port:** `12346`



8. Continue the training procedure as always (continue from Step 3.3.10).
9. Unity will then receive the updated jitter value for the channel configured in `X+`.

How to debug the side detection

The side detection system (left, front, or right) is already supported by the classification pipeline, although it is not currently used in the demo for simplicity. If you want to use side detection for developing a new application, you can experiment with different training parameters to improve the results.

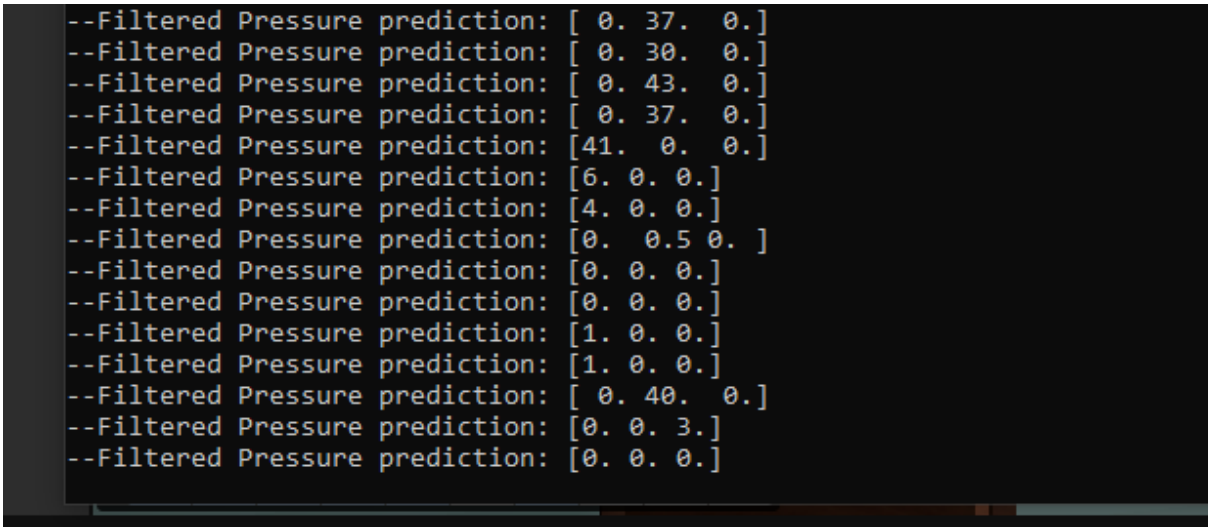
The recommended first step is to perform a full training cycle and test the demo directly on the computer in Unity instead of on the VR headset. To do this:

1. Perform the training procedure as explained in Section 3. The difference will be that, when writing the Quest IP (step 3.3.12), **you should write the IP of your own computer instead** (for example, 127.0.0.1).
3. Open the Unity project and press Play.
5. In the **Console**, verify whether the tongue movements you perform match the predicted pressure values. Data is logged as (jitter, left pressure [%], front pressure [%], right pressure [%]).
6. If the predictions do not match your intended movements, try retraining the model using slightly different tongue gestures or positions that feel more comfortable for you.

It is also possible to increase the number of training trials, modify filtering options or add additional labels. To do this, modify the scripts `CLASSIFICATION_PIPELINE.py`, `Model_trainer.py` and `Online_classification.py` in `SmartRoom/Pressure_Classifier`.

How to debug the gesture detection without using the MetaQuest

It is possible to debug the gesture detection without using the Meta Quest or even starting Unity. To do this, follow all the steps in Section 3 to start the online classification. You do not need to open Unity or use the Meta Quest. Just go to the Python console and pay attention to the numbers that are printed. You will see something like this:



The numbers represent the percentage of tongue pressure in the left, front, and right directions:

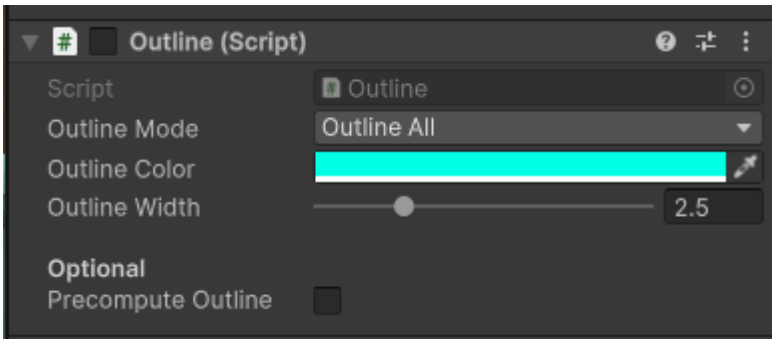
`[left pressure [%], front pressure [%], right pressure [%]]`

Try to go to the resting position. You should see numbers between 0 and 10. Then try performing gestures on the left, right, and front, and check if the numbers change accordingly.

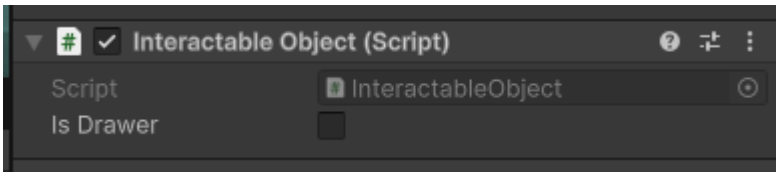
How to add a new interactable object

To add a new interactable object to the T-Press demo in Unity, follow these steps:

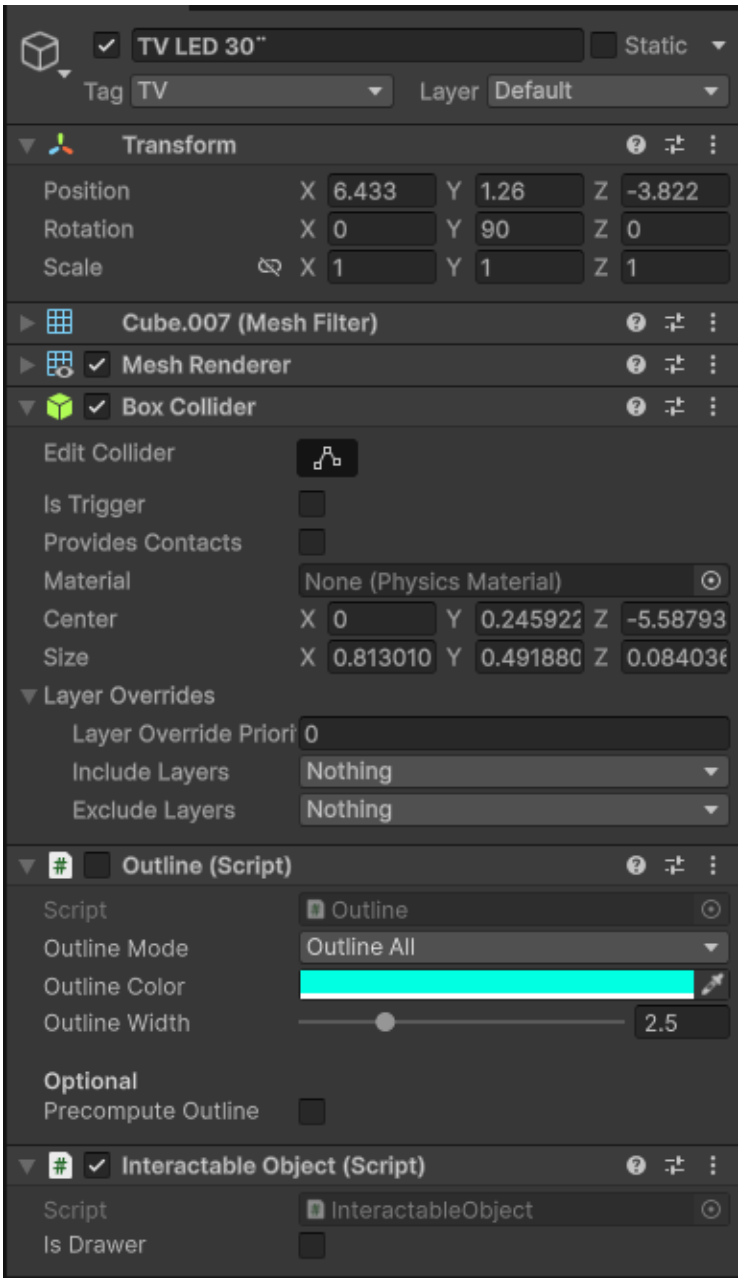
1. Add the new object to the scene. For example, an air conditioner will be used in this example.
2. Add the `Outline (Script)` component to the air conditioner object and set the outline color to light blue. Then, disable the component. You can also copy this component from the GameObject called `TV LED` by right-clicking the component and selecting **Copy Component**, then right-clicking on the new object and selecting **Paste Component As New**.



3. Add the `InteractableObject.cs` script to the air conditioner.



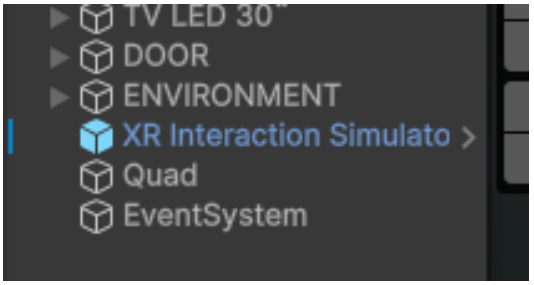
4. Make sure the object also contains a `Box Collider` component so it can be detected by the raycast system.
5. The final object setup should include (see picture below):
    - Mesh or model
    - `Box Collider`
    - `Outline (Script)`
    - `InteractableObject.cs`
6. Set the tag of the object to represent its interaction type. Since the air conditioner is not included in the demo by default, create a new tag called `aircon`.
7. Open the script `InteractionManager.cs` and add the interaction logic for objects with the `aircon` tag. This is where you define what happens when the user selects the air conditioner with a tongue gesture.
8. Open the script `HeadRaycast.cs` and add the new tag `aircon` to the collider comparison. See picture below.




```
if (hit.collider.CompareTag("Light") ||
    hit.collider.CompareTag("Drawer") ||
    hit.collider.CompareTag("TV") ||
    hit.collider.CompareTag("Door") ||
    hit.collider.CompareTag("aircon"))
{
    currentColor = validHitColor;
    currentMaterial = blue_mat;
}
```

Using the XR Interaction Simulator

To debug the Unity scene, it is recommended to enable the **XR Interaction Simulator**. This prefab allows you to move around the environment with WASD and rotate the head by maintaining the right mouse button. If you use this utility, remember to **disable it** before deploying to the headset.



8. Future research directions

This demo is only a simple demonstration of the capabilities of tongue-based interaction, but its potential can be expanded much further. The following are some suggestions for future work that could use this demo as a starting point.

1. **More tongue interactions**. Expand the code in Unity to support single clicks, double clicks, and sustained clicks. This will allow users to perform more complex actions.
2. **Smart house**. Create additional smart environment interactions, such as a smart kitchen or smart garden. What everyday activity would fit the interaction?
3. **Augmented Reality**. Adapt the interaction system to AR environments in order to interact with real-world devices instead of only virtual objects.
4. **Walking interaction**. Investigate how the device performs wirelessly while the user is walking. Since the current tests were performed with participants seated, further research could explore whether motion requires additional signal processing techniques.
5. **Include side detection**. The tongue detection system is capable of detecting tongue direction (left, front, right), although side detection is not currently used in the demo for simplicity. Future work could explore using the sides for different tasks, for example, sorting boxes in different containers depending on the tongue position. Although this may seem challenging, preliminary work on this topic has already been conducted, even if it was not formally published. Read Section 7 to learn how to debug the side detection.
6. **UI interaction**. Explore the use of tongue interaction for everyday UI interaction, such as navigating menus, watching videos, listening to music, etc.
7. **Soft wearable**. Create a soft-wearable for easy sensor placement.

9. Troubleshooting

Noisy signal in OpenBCI

1. Check the sensor placement. Are any sensors detached? Is the user sweating? Because of the chin shape, it can be difficult to place the sensors correctly on some users. Keep in mind that if you change the sensor placement, you need to retrain the model.

2. If a lot of noise is being detected, especially on the right side of the graph in OpenBCI GUI, try moving the board to another surface that is relatively far from you. For example, place it on a chair next to you.

3. Is your phone next to the cables or the board? If so, move your phone and any other electronic devices away.

4. Are you in a room with many electronic devices? If so, try moving to another place.

Battery is too difficult to insert in the board

Because of the short cable, the battery can be difficult to insert into the board, especially during user tests when it is removed many times for charging. To solve this, you can use a battery with longer cables or solder a cable extension yourself.

10. Attributions

Used packages:
TV LED 30" | 3D Electronics | Unity Asset Store
Low Poly Bedroom Asset Pack | 3D Urban | Unity Asset Store
Quick Outline | Particles/Effects | Unity Asset Store
Daily News Theme | Royalty-free Music - Pixabay
Free Wood Door Pack | 3D Interior | Unity Asset Store